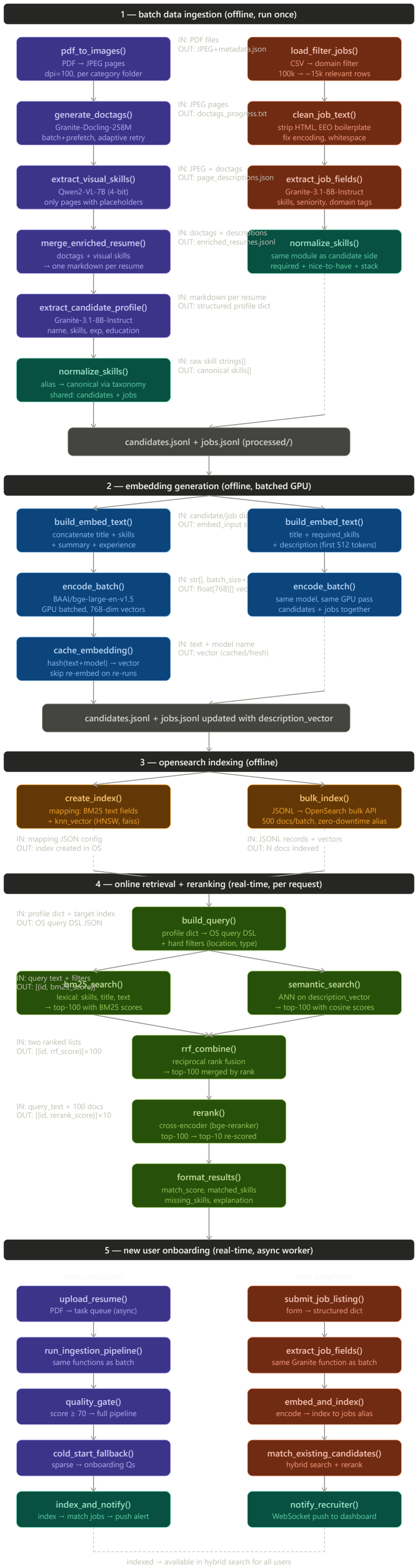
1 — batch data ingestion (offline, run once)
pdf_to_images()
PDF → JPEG pages
dpi=100, per category folder
IN: PDF files
OUT: JPEG+metadata.json
load_filter_jobs()
CSV → domain filter
100k → ~15k relevant rows
generate_doctags()
Granite-Docling-258M
batch+prefetch, adaptive retry
IN: JPEG pages
OUT: doctags_progress.txt
clean_job_text()
strip HTML, EEO boilerplate
fix encoding, whitespace
extract_visual_skills()
Qwen2-VL-7B (4-bit)
only pages with placeholders
IN: JPEG + doctags
OUT: page_descriptions.json
extract_job_fields()
Granite-3.1-8B-Instruct
skills, seniority, domain tags
merge_enriched_resume()
doctags + visual skills
→ one markdown per resume
IN: doctags + descriptions
OUT: enriched_resumes.jsonl
normalize_skills()
same module as candidate side
required + nice-to-have + stack
extract_candidate_profile()
Granite-3.1-8B-Instruct
name, skills, exp, education
IN: markdown per resume
OUT: structured profile dict
normalize_skills()
alias → canonical via taxonomy
shared: candidates + jobs
IN: raw skill strings[]
OUT: canonical skills[]
candidates.jsonl + jobs.jsonl (processed/)
2 — embedding generation (offline, batched GPU)
build_embed_text()
concatenate title + skills
+ summary + experience
IN: candidate/job dic
OUT: embed_input s
build_embed_text()
title + required_skills
+ description (first 512 tokens)
encode_batch()
BAAI/bge-large-en-v1.5
GPU batched, 768-dim vectors
IN: str[], batch_size=
OUT: float[768][] vec
encode_batch()
same model, same GPU pass
candidates + jobs together
cache_embedding()
hash(text+model) → vector
skip re-embed on re-runs
IN: text + model name
OUT: vector (cached/fresh)
candidates.jsonl + jobs.jsonl updated with description_vector
3 — opensearch indexing (offline)
create_index()
mapping: BM25 text fields
+ knn_vector (HNSW, faiss)
bulk_index()
JSONL → OpenSearch bulk API
500 docs/batch, zero-downtime alias
IN: mapping JSON config
OUT: index created in OS
IN: JSONL records + vectors
OUT: N docs indexed
4 — online retrieval + reranking (real-time, per request)
IN: profile dict + target index
OUT: OS query DSL JSON
build_query()
profile dict → OS query DSL
+ hard filters (location, type)
IN: query text + filters
OUT: [{id, bm25_score}]
bm25_search()
lexical: skills, title, text
→ top-100 with BM25 scores
semantic_search()
ANN on description_vector
→ top-100 with cosine scores
IN: two ranked lists
OUT: [{id, rrf_score}]×100
rrf_combine()
reciprocal rank fusion
→ top-100 merged by rank
IN: query_text + 100 docs
OUT: [{id, rerank_score}]×10
rerank()
cross-encoder (bge-reranker)
top-100 → top-10 re-scored
format_results()
match_score, matched_skills
missing_skills, explanation
5 — new user onboarding (real-time, async worker)
new candidate
new employer
upload_resume()
PDF → task queue (async)
submit_job_listing()
form → structured dict
run_ingestion_pipeline()
same functions as batch
extract_job_fields()
same Granite function as batch
quality_gate()
score ≥ 70 → full pipeline
embed_and_index()
encode → index to jobs alias
cold_start_fallback()
sparse → onboarding Qs
match_existing_candidates()
hybrid search + rerank
index_and_notify()
index → match jobs → push alert
notify_recruiter()
WebSocket push to dashboard
indexed → available in hybrid search for all users